# STA 561 Final Project Report

Cael Elmore, Cokie Parker, Pranavi Reddi, Ananya Yadav, Andrew Wang

April 4 2026

## 1 Introduction

The New York Times Connections game is a category-matching puzzle, released daily by editor Wyna Liu. Launched in 2023, Connections has become one of the most popular daily puzzle games worldwide and is the second most popular New York Times game, behind Wordle.

Designed by professional puzzle makers, each game consists of a four-by-four grid of sixteen separate words. The player must find 4 hidden groupings of 4 words. Each such category is color-coded by straightforwardness. Color rankings are yellow, green, blue, and purple, with yellow as the most straightforward, or easiest. Within a single puzzle, categories can also overlap ("Red herrings", or groupings of less than 3 or more than 4 feasibly similar words), adding additional subtlety to puzzle creation.

A number of open-source solvers for this puzzle exist, but few generators. This project creates a programmatic generator of puzzles. Our full codebase can be found here: `https://github.com/CokieLee/ProbML-Connections-Project`

Our playable puzzle can be found here: `https://prob-ml-connections-project.vercel.app/index.html`

### 1.1 General Strategy for Generating Categories and Words

Our approach generally involved pre-generating large lists of possible word groupings for each color ranking, and then assembling a final puzzle by selecting 4 such groupings that give appropriate overlap (or "red herrings").

For the yellow category, (typically synonyms in real puzzles) we generated categories of related words using semantic embeddings. For this section, we selected words from databases filtering for clarity, familiarity, and prioritizing consistency within the group.

Green, which we classified as moderately difficult, built categories based on recognizable relationships, such as "types of," "parts of," or concepts that are closely related. We combined curated examples with automatically extracted word relationships (from structured word relationship databases), somewhat increasing difficulty and thought required for solving this category.

Blue, classified as a "hard" category, typically involves trivia and more subtle patterns. In generating this category, we focused on pronunciation patterns and linguistic quirks between terms, and relied on the CMU phonetic dictionary and rules-based generation. We did previously attempt trivia and pop culture based categories, but discarded this idea after struggling to get an appropriate obscurity level balance.

Finally, purple, the hardest category, is often based on patterns in the structure of words rather than their meanings or definitions. The purple category almost always contains multi-step connections or "aha" moments before the solution becomes clear. In generating these categories, we relied primarily on a rules-based generation of pre-defined categories, such as "Words that are a new word when one letter is removed", or "words that are all prefixes/suffixes in another word".

Finally, to make the product work with a frontend, we precomputed and stored all candidate categories ahead of time (for better speed performance), for on-demand assembly when a user refreshes the front-end. The frontend site we developed is made to look and act just like the user interface of the NYT Connections game, and also reveals the answer after 4 failed attempts.

## 1.2 Results

To evaluate the quality of our generated puzzles, we implemented the embedding-based solver described by Merrill (2024), which uses GloVe word embeddings to greedily cluster words by semantic proximity. The algorithm works by selecting a seed word, iteratively adding the word whose embedding vector is closest to the running cluster centroid, and repeating until a group of four is formed. The tightest cluster across all possible seeds is selected as the solver's first guess, those four words are removed, and the process repeats for the remaining twelve, then eight, then four. We ran this solver on 300 generated puzzles assembled from our processed_categories.json pipeline and on 300 randomly sampled real NYT Connections puzzles drawn from the Eyefyre/NYT-Connections-Answers archive. In parallel, we ran a second implementation of the same greedy algorithm using sentence-transformer embeddings (Reimers and Gurevych, 2019), specifically the all-MiniLM-L6-v2 model, which produces 384-dimensional contextual embeddings trained on over one billion sentence pairs. This allowed us to compare a classical distributional embedding model against a modern transformer-based one on the same task.

The GloVe solver correctly identified an average of 0.71 out of 4 groups on our generated puzzles, compared to 0.37 on real NYT puzzles. The sentence-transformer solver achieved 0.99 on generated puzzles and 0.34 on NYT puzzles. A two-sample Kolmogorov-Smirnov test confirmed that the solve distributions are significantly different between generated and NYT puzzles for both the GloVe solver ($KS = 0.233$, $p < 0.0001$) and the sentence-transformer solver ($KS = 0.427$, $p < 0.0001$). These results indicate that our generated puzzles are, on average, easier for an embedding-based solver to crack than real NYT puzzles. However, the magnitude of the difference is modest in practical terms.

On our generated puzzles, the GloVe solver fails to identify any group correctly 48.0% of the time, compared to 71.3% on NYT puzzles. Both distributions are heavily concentrated at zero, which means the solver struggles with the majority of puzzles in both sets. The difference is driven primarily by the tail of the distribution, where 15.3% of generated puzzles yield two correct groups versus only 5.0% for NYT. The per-tier analysis reveals that our difficulty gradient is directionally correct and internally consistent. On generated puzzles, the GloVe solver identifies the yellow (easiest) category 40.0% of the time, green 26.7%, blue 3.3%, and purple 1.3%. This monotonic decrease from yellow to purple mirrors the pattern observed in NYT puzzles, where the solver identifies yellow 15.0% of the time, green 12.7%, blue 5.7%, and purple 3.3%. The gradient is steeper in our generated puzzles than in NYT, which suggests that our yellow and green categories may be too semantically tight relative to blue and purple. In other words, the easy tiers are too easy, rather than the hard tiers being too easy. This is consistent with the incoherence analysis, which measures how dispersed a category's word vectors are in embedding space. Our yellow categories have a mean incoherence of 3.548, compared to 3.635 for NYT yellow. Lower incoherence means the words cluster more tightly, making them easier for the solver to identify. At the purple tier, our categories have a mean incoherence of 4.360 versus 4.080 for NYT, suggesting that our purple categories are actually less coherent in embedding space than real NYT purple categories and therefore harder for an embedding-based solver to group correctly.

The category type breakdown provides insight into which of our generators produce the most challenging categories. The letter drop and two-layer rhyme generators, which require the solver to recognize structural wordplay rather than semantic relatedness, have the lowest solve rates. The GloVe solver identifies the correct group only 51.5% of the time for letter drop categories and 50.0% for two-layer rhyme categories, compared to 81.2% for homophone categories and 80.0% for parts-of categories. This makes intuitive sense. An embedding-based solver operates entirely in semantic space, so categories defined by meaning-based relationships (homophones, hypernyms, meronyms) are precisely the categories it is designed to detect. Categories defined by letter-level structural patterns (anagrams, hidden words, letter drops) have no reason to cluster in embedding space, which is why the solver struggles with them. This is exactly the property that makes them good purple categories. They are invisible to the dominant solving strategy.

The solver agreement analysis offers an additional perspective. On NYT puzzles, the GloVe and sentence-transformer solvers agree 63.7% of the time, with GloVe performing better 19.0% of the time and the LLM performing better 17.3% of the time. On our generated puzzles, agreement drops to 44.7%, and the LLM outperforms GloVe in 39.0% of cases compared to 16.3% where GloVe wins. This asymmetry suggests that our generated puzzles contain semantic structure that the sentence-transformer can exploit more effectively than GloVe. The likely explanation is that our yellow and green categories, which are built from WordNet synsets and hyponym trees, produce word groups with strong contextual relationships that a transformer trained on sentence-level objectives

can detect but that a word-level co-occurrence model like GloVe cannot. This finding points toward a concrete avenue for improvement. If we want to increase difficulty against modern solvers, we should select yellow and green categories that have higher incoherence in sentence-transformer space specifically, not just in GloVe space.

Taken together, these results suggest that our puzzle generator produces output that is qualitatively similar to real NYT Connections puzzles but systematically easier at the yellow and green tiers. The purple tier, which is generated entirely through algorithmic wordplay rather than semantic grouping, is appropriately difficult and in some cases harder than real NYT purple categories as measured by embedding incoherence. The primary path to closing the difficulty gap is to introduce more misdirection at the easier tiers, either by selecting words with higher polysemy that could plausibly belong to multiple categories or by deliberately choosing categories whose members are semantically distant from one another in embedding space while still sharing a valid grouping criterion.

## 1.3 Future Work

A possible extension of our work involves improving the red-herring creation system. Currently, our system involves picking the purple category, and then selecting categories with the most similar embeddings to this initial embedding for the remaining categories, by performing a clustering. This sometimes leads to insufficient overlap between categories, especially when niche terms or references are present. On the flip side, it sometimes leads to categories that are too similar and cannot be reasonably distinguished (i.e. when one category is related to shoes, and another is related to feet, and both categories contain words having to do with both shoes and feet). One way we could make the red-herring feature of our final puzzle assembly step more sophisticated, would be to select subsequent puzzles by their variability of embedding similarity to our starting category, rather than net similarity. This way, we ensure that only some, not all, words in subsequent categories have an overlap with our starting puzzle, preventing ambiguous puzzles.

Additionally, as suggested in the results section, we could improve our similarity with NYT puzzles by increasing the difficulty of our yellow and green categories (at least with respect to our simple semantic solver), or by prioritizing relationships which might be more intuitive to humans than a semantic solver. For example, embeddings may be based on co-occurence in a sentence, but sometimes, words which are related might not be very likely (indeed, may be often mutually exclusive in a sentence, such as when talking about types of coffee, for example) to co-occur. To address this issue, we could start by adding more rule-based constraints, such as that for some percentage of categories, all words in the category must be the same part of speech.

## 2 Frequently Asked Questions

1. **How are the puzzles created?**

   The puzzles are built from category banks across four difficulty colors: yellow, green, blue, and purple. Each color reflects a different style of connection, from more direct relationships to more unusual or tricky ones.

2. **Are the puzzles written by hand or generated automatically?**

   The puzzles are generated using a structured pipeline that searches across category libraries, compares relationships between candidate groups, and filters out bad combinations such as repeated words or overly messy overlaps.

3. **How are bad puzzles filtered out?**

   Bad candidates are removed if they contain repeated words, too much lexical overlap, too much semantic bleed between categories, or category combinations that are too broadly similar across the board.

4. **In clustering, why did you normalize the components in the hybrid vector before combining?**

   The label embedding encodes a phrase while the centroid encodes the average of four word vectors, so these can point in quite different directions. If combined by simple averaging without re-normalization, the result is not a unit vector and cosine similarity comparisons become inconsistent. If concatenated, the dimensionality triples and the inner product geometry changes entirely.

5. **Why do some puzzles feel harder than others?**

   Even within the same general framework, some combinations naturally produce clearer boards while others produce more deceptive ones. Difficulty depends on category familiarity, word ambiguity, and how much cross-category temptation exists.

6. **Isn't there structurally bias in the puzzle because the generation always starts with a purple category?**

   By always anchoring the ambiguity to purple, we guarantee that at least one misleading pair exists in each puzzle, but we do not generate puzzles where the primary trap involves, say, confusing a green 'parts of X' category with a yellow 'semantic cluster' category. In the real NYT Connections, the most devilish puzzles sometimes hide the trap in unexpected color pairs, not just at the purple level. Our approach is a reasonable first-order approximation of the NYT style but is more formulaic. A richer extension would select the ambiguous neighbor pair from any two colors, with purple weighted more heavily rather than exclusively.

7. **How does the system decide which categories go together?**

   The system compares categories in an embedding space, which helps estimate semantic similarity. It also uses additional rules to reduce bad matches, avoid repeated words, and prevent one category from being too closely related to too many others.

8. **Why not just pick the closest categories?**

   Because that often makes bad puzzles. A good puzzle needs some ambiguity, but not too much. The goal is to create boards where categories feel plausible and interesting without becoming unfair or collapsing into multiple equally valid solutions.

9. **How does the algorithm avoid selecting duplicate words?**

   Any candidate board that does not have exactly 16 distinct words is rejected outright. Additional checks also penalize things like:

   (a) morphological overlap (e.g., ENTER / ENTERING)

   (b) substring overlap (e.g., RAIN / RAINBOW)

10. **How did you choose K, and what guarantees do you have that the clusters are semantically meaningful?**

    K-Means minimizes within-cluster variance in Euclidean space, but cosine similarity (which sentence transformers are optimized for) is not Euclidean distance. Clusters that look tight under L2 may span broad semantic ranges under cosine, and vice versa. We selected K empirically by inspecting sample cluster outputs for coherence. Any reader should treat cluster-derived categories as promising candidates subject to the scoring filter, not guaranteed semantic groups.

11. **Doesn't the 9 pronunciation pattern buckets create a very small space for the blue category generation? What stops many generated puzzles from reusing the same bucket?**

    If the candidate pool within a sparse bucket has fewer than 4 suitable words, that bucket simply cannot generate a valid group. We do not currently enforce that any given bucket appears at most once across a session's puzzles, so repeat themes are possible.

12. **WordNet meronyms are notoriously incomplete and inconsistently annotated, so how do you handle these gaps in the green category generation?**

    Our generator samples from whatever meronyms WordNet provides, which means the quality of a 'parts of X' category is directly tied to the completeness of that node's annotation. We do apply frequency filtering to remove obscure lemmas even when WordNet includes them, but we do not have a mechanism to detect when a meronym list is simply incomplete rather than genuinely short. Practically, this means categories built

on well-annotated synsets are reliable; categories built on sparsely annotated ones may include odd ones.

13. **Can the system generate unlimited puzzles?**

    In principle, it can generate many puzzles, but quality matters more than raw quantity. The system works best when it builds a large pool of candidates and keeps only the strongest ones.

14. **How do you deal with category of words that all appear near 'the' and 'of' and score high for spurious reasons, since the cosine similarity the yellow scoring function depends on measures distributional similarity between word embeddings?**

    Sentence-transformer embeddings do reflect distributional co-occurrence and are not pure measures of semantic category membership. Very common words cluster together not because they mean similar things but because they appear in similar syntactic positions. Our filter against obscure terms (via frequency thresholds) partially mitigates this. The WordNet-sourced categories serve as a structural backbone that grounds the semantic claims in a linguistic hierarchy rather than pure distributional statistics.

15. **Is it possible for a generated puzzle to copy an existing puzzle exactly?**

    No. It is inspired by the general idea of grouping words by hidden relationships, but the generation process, category libraries, and validation logic are part of its own system. There is also a check in place to make sure no puzzle is displayed if a previous NYT connections puzzle has the exact same words and category groupings.

# 3 Technical Appendix

"A technical appendix (no length requirement). This will be the bulk of your submission. This will include all technical details, code, and results. I must be able to reproduce your results from only the technical description and links to raw datasets. Where appropriate, include a demo in a jupyter notebook."

## 3.1 Yellow words

1. **Code** A .json file of yellow difficulty grouped words can be produced by running final_yellow_puzzle_generation.py.

2. **External Data and Code Sources** The system relies on the following external datasets and pre-trained models:

   - **WordNet (NLTK):** A large lexical database of English nouns, used as the primary source for semantic hierarchies (synsets and hyponyms).
   - **Hugging Face (`all-MiniLM-L6-v2`):** A sentence-transformer model that maps words into a 384-dimensional vector space. It is used to calculate semantic similarity.
   - **Zipf Frequency (via `wordfreq`):** A dataset providing word frequency counts across large corpora, used to ensure words are common enough for general puzzles.

3. **Core Concepts and Definitions**

   - **Zipf Frequency:** The frequency of a word is measured on a logarithmic scale. The Zipf value $z$ is defined as $z = \log_{10}(\text{frequency per billion words}) + 9$. A value of 3.0 represents 1 occurrence per million words; the script filters for $z \geq 3.6$ to ensure words are common enough for a general puzzle audience.
   - **Cosine Similarity:** The semantic similarity between two word vectors $\mathbf{A}$ and $\mathbf{B}$ is calculated as:

     $$\text{sim}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

     Since vectors are normalized during embedding, this simplifies to the dot product.
   - **Jaccard Similarity:** Used to ensure novelty between generated groups. For sets $A$ and $B$, it is computed as:

     $$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

4. **Step 1: Vocabulary Construction** The system builds a candidate vocabulary by iterating through all noun synsets in WordNet.

- **Normalization:** Words are lowercased, stripped of underscores, and filtered to include only alphabetic characters.
- **Filtering:** Words must have a length $L \in [3, 12]$ and a Zipf frequency $\geq 3.6$.
- **Output:** A sorted list of the top 12,000 most common valid nouns.

5. **Step 2: Vector Embedding** Each word in the vocabulary is passed through the `all-MiniLM-L6-v2` transformer. The resulting embeddings are $L2$-normalized such that the magnitude of each vector is 1.0. This allows semantic similarity to be computed efficiently via linear algebra.

6. **Step 3: Category Bank Generation** The pipeline aggregates potential categories from three distinct sources:

   - **WordNet Hierarchy:** For every noun synset, the system collects all its immediate hyponyms (sub-types). If the resulting group of members (after vocabulary filtering) size is between 6 and 30, it is added to the bank.
   - **K-Means Clustering:** To find semantic groups not strictly defined by WordNet, the script performs MiniBatchKMeans clustering on the embedding space. The number of clusters $k$ is dynamically set based on vocabulary size ($k \approx$ vocab_size/8). Clusters sized between 6 and 18 are retained.
   - **Manual Templates:** A small set of high-quality "seed" categories (e.g., colors, fruits) are included to ensure baseline coverage.

7. **Step 4: Group Sampling and Semantic Scoring** For each category in the bank, the system samples combinations of 4 words ($C(n, 4)$). Each group is assigned a "Yellow Score" $S$ calculated as follows:

   $$S = 0.42(\mu_{sim}) + 0.18(\min_{sim}) + 0.28\left(\frac{\mu_{freq}}{8}\right) + 0.12\left(\frac{\min_{freq}}{8}\right) - 0.10(\sigma_{sim})$$

   Where:

   - $\mu_{sim}$: Mean pairwise cosine similarity of the 4 words.
   - $\min_{sim}$: The lowest similarity between any two words in the group.
   - $\mu_{freq}$: Mean Zipf frequency of the group.
   - $\sigma_{sim}$: Standard deviation of pairwise similarities (penalizing groups with "outlier" words).

   Groups must achieve a minimum score (default 0.44) to be considered candidates.

8. **Step 5: Novelty and Deduplication** To prevent repetitive puzzles, a `NoveltyMemory` class tracks all accepted groups. A new candidate group is rejected if:

- Its Jaccard similarity with any existing group is $\geq 0.50$.
- It shares the same label as a previous group and has a Jaccard similarity $\geq 0.25$.
- It contains 3 or more words that have already been used for that specific category label.

9. **Step 6: Iterative Relaxation and Final Output** The generation runs in rounds. If the target count (2,500 groups) is not met, the system "relaxes" the minimum score threshold by 0.01 every 8 rounds (down to a floor of 0.39). This ensures the pipeline produces a large, diverse dataset even if strict semantic coherence criteria are difficult to meet for rarer categories. The final result is exported as a JSON object containing the words, category labels, and the calculated technical metrics.

## 3.2 Green words

**1. Code**
A .json file of green difficulty grouped words can be produced by running GREEN_GENERATOR.py.

**2. External Data Sources and Dependencies**
To recreate this system, the following external datasets and libraries are required:

- **WordNet (via NLTK):** A large lexical database of English where nouns, verbs, adjectives, and adverbs are grouped into sets of cognitive synonyms (*synsets*).
- **Wordfreq:** A Python library that provides access to the **Zipf scale** of word frequency, derived from the Exquisite Corpus (including Wikipedia, subtitles, and web crawls).
- **NLTK Corpora:** Specifically `wordnet` and `omw-1.4`.

**3. Fundamental Concepts and Definitions**

- **Zipf Frequency:** This is a logarithmic measure of word frequency. Unlike raw counts, the Zipf scale typically ranges from 0 to 8. It is calculated as:

$$\text{Zipf} = \log_{10}(\text{frequency per billion words}) + 3$$

  For example, a Zipf score of 3.0 means the word appears once per million words. The script uses a `MIN_ZIPF` threshold (default 3.0) to ensure puzzle words are "common" enough for a general audience.

- **Synsets and Lemmas:** In WordNet, a *synset* represents a specific concept (e.g., "bass" as a fish vs. "bass" as a guitar). A *lemma* is the specific word form associated with that synset.

- **Semantic Relations:**
  - *Hyponyms:* "Specifics" of a word (e.g., "Salmon" is a hyponym of "Fish").
  - *Hypernyms:* "Generics" of a word (e.g., "Fish" is a hypernym of "Salmon").
  - *Meronyms:* Parts of a whole (e.g., "Wheel" is a meronym of "Car").
- **Polysemy Count:** The number of distinct synsets associated with a word. Higher polysemy indicates higher ambiguity.

**4. Technical and Algorithmic Steps**
The generation process follows a linear pipeline:

**Step A: Lexical Filtering (The `is_good_word` logic)**
Before any category is formed, words are passed through a strict filter to ensure "clean" results:

1. **Morphology:** Exclude words containing spaces, underscores, or hyphens (multi-token entries).
2. **Character Set:** Must match the regular expression `[A-Za-z]+`.
3. **Length:** Must fall within `MIN_LEN` (3) and `MAX_LEN` (12).
4. **Commonality:** Must have a Zipf frequency $\geq 3.0$.

**Step B: Category Generation Engines**
The script populates a "Category Bank" using seven distinct methods:

1. **Manual Banks:** Hard-coded lists for high-quality categories (e.g., "Musical Instruments", "Programming Languages").
2. **Color Samples:** Random combinations of common color words.
3. **Synonym Mining:** Iterating through WordNet synsets and extracting 4 lemmas that pass the Zipf filter.
4. **Taxonomic ("Types of"):** Selecting a parent synset (e.g., "Fruit") and gathering its recursive hyponyms (e.g., "Apple", "Pear").
5. **Sister Hyponyms:** Selecting a parent and gathering words that share that immediate parent (siblings).
6. **Meronymy ("Parts of"):** Extracting part, substance, or member meronyms for a given synset.
7. **Suffix Patterns:** Identifying words that share a common ending (e.g., -ER, -ING, -IST).

**Step C: Scoring and Ranking**
Each generated group is assigned a **Quality Score**, calculated as the arithmetic mean of the Zipf frequencies of the four words:

$$\text{Score} = \frac{1}{4}\sum_{i=1}^{4} \text{Zipf}(w_i)$$

Groups are then ranked by this score, with additional "Label Bonuses" (0.10 to 0.25) added to manual or taxonomic types to prioritize them over raw WordNet synonyms.

**Step D: Deduplication and Relatedness Filtering**
To prevent "muddy" puzzles, the system performs a collision check:

1. **Canonical Grouping:** Words are normalized (lowercase), sorted, and converted to a tuple to ensure `(A, B, C, D)` is treated the same as `(D, C, B, A)`.

2. **Neighbor Overlap:** For any two categories, the script collects "semantic neighbors" (synonyms, hypernyms, and hyponyms of all words). If the intersection of these neighbor sets is $\geq 3$, the categories are deemed too related and cannot appear in the same puzzle.

**5. User-Defined Parameters**

| Parameter | Default | Description |
|---|---|---|
| `MIN_ZIPF` | 3.0 | Lower bound for word commonality (3.0 = 1 per million). |
| `MAX_POLYSEMY` | 8 | Maximum number of definitions a word can have in WordNet. |
| `MIN_DEPTH` | 4 | Minimum depth in the WordNet hierarchy to avoid abstract parents like "Entity". |
| `MAX_TRIES` | 50,000 | The number of random sampling attempts to find 4 non-colliding categories. |

## 3.3 Blue words

1. **Code**
   A .json file of blue difficulty grouped words can be produced by running blue_PhoneticSimilarity_generator.py.

2. **External Data Sources and Dependencies**
   The following external resources are strictly required for the script to function. Users must ensure these specific datasets are accessible to the Python environment:

   - **CMU Pronouncing Dictionary (CMUdict):**

- **Source:** Carnegie Mellon University via NLTK Data.
- **Function:** Provides the mapping of English words to ARPAbet phonemes.
- **Access:** Retrieved via `nltk.download('cmudict')`.

- **English Words (web2):**
  - **Source:** `english-words` Python package (based on Webster's Second New International Dictionary).
  - **Function:** Serves as the base corpus for "NYT-appropriate" common vocabulary.
  - **Access:** Retrieved via `from english_words import get_english_words_set`.
- **NLTK Library:**
  - **Function:** Provides the computational framework for processing lexical resources.

3. **Resource Loading and Pre-processing**
   The system utilizes the NLTK `cmudict` as the ground truth for phonetic transcriptions. Let $W$ be the set of words from the `web2` English dictionary. The subset of candidate words $C$ is filtered such that:

$$C = \{w \in W : w \in \text{cmudict} \wedge 3 \leq \text{len}(w) \leq 10\}$$

4. **Heuristic Detection of Phonetic Anomalies**
   For each word $w \in C$, the script computes a phoneme list $P_w$. We define a "Silent Letter" function $S(w, l, \phi)$ where $l$ is a grapheme and $\phi$ is the expected phoneme:

$$S(w, l, \phi) = \begin{cases} \text{True} & \text{if } l \in w \wedge \phi \notin P_w \\ \text{False} & \text{otherwise} \end{cases}$$

   This is applied to 'P', 'K', 'B', 'L', and 'H' (at the start of words).

5. **Complex Phonetic Mapping**
   Specialized logic is applied for non-silent similarities:

   - **Ending 'QUE':** A word $w$ is collected if it ends in the string 'que' and the terminal phoneme in $P_w$ is $/K/$.
   - **Voiced Postalveolar Fricative (/ZH/):** Criteria: $(ZH \in P_w) \wedge ('s' \in w) \wedge ('z', 'j' \notin w)$.
   - **Long I 'Y':** Criteria: $('y' \in w) \wedge (AY \in P_w)$.

6. **Iterative Grouping Algorithm**
   The algorithm populates buckets $B_1, B_2, \ldots, B_n$ corresponding to these rules. The final output is generated via:

   (a) Define target iterations $T = 2500$.

(b) While count $< T$:

(c) Select a random bucket $B_i$ where $|B_i| \geq 4$.

(d) Sample $\{w_1, w_2, w_3, w_4\} \subset B_i$.

(e) Store as a valid puzzle grouping.

7. **Pre-processing**
All phonemes are normalized by removing lexical stress markers (integers 0-2). To recreate this, ensure `nltk.download('cmudict')` and `pip install english-words` are executed in the local environment.

8. **Setup and Execution Protocol**
To reproduce the phonetic groupings from scratch, follow these steps in a standard Python 3.x environment:

   (a) **Dependency Installation:** Install the necessary libraries using the Python package manager:

   ```
   pip install nltk english-words
   ```

   (b) **Resource Initialization:** The script automatically handles the download of the CMU Dictionary upon its first run using `nltk.download('cmudict')`.

   (c) **Script Invocation:** Execute the script from the terminal or command line:

   ```
   python blue_PhoneticSimilarity_generator.py
   ```

   Upon successful completion, the script will output a message: *"Successfully generated [N] groupings."* and generate a file named `phonetic_connections.json` in the root directory. This JSON file contains the finalized groups of four words categorized by their shared phonetic anomaly.

## 3.4 Purple words (Suffix and Prefix Pattern)

**1. Code Location and Environment**

- **File Name:** `purple_SuffixPrefix_generator.ipynb`
- **Execution Environment:** Python 3.9+ via Jupyter Notebook or standard interpreter.

**2. External Data Sources and Dependencies** To recreate this system, the following libraries and data corpora are required:

- **wordfreq (v3.1.1):** Provides access to the *Linguistic Data Consortium* and *Google Books Ngram* data to estimate word prevalence.
- **pronouncing (v0.2.0):** A wrapper for the *CMU Pronouncing Dictionary*, used for phonetic rhyme discovery.
- **re & collections:** Standard libraries for regex-based cleaning and data structuring.

**3. Core Linguistic Metrics and Definitions** The algorithm relies on the **Zipf Scale** to ensure words are recognizable to players.

- **Zipf Frequency ($z$):** Computed as the base-10 logarithm of the number of occurrences per billion words.

$$z = \log_{10}(\text{occurrences per billion}) + 1$$

  A value of 3.0 indicates a word appears approximately once per million words.

- **Minimum Zipf Clue (MIN_ZIPF_CLUE = 3.0):** The threshold for individual clue words (e.g., "MARK").
- **Minimum Zipf Compound (MIN_ZIPF_COMPOUND = 2.5):** The threshold for the resulting compound word (e.g., "MARKSMAN").

**4. Step-by-Step Algorithmic Process** **Step A: Vocabulary Initialization**

1. Extract the top $n$ $(100,000)$ words for the English language (`en`) from `wordfreq`.
2. **Token Cleaning:** For every word $w$, apply the transformation:

$$f(w) = \text{lowercase}(w) \setminus \{c \mid c \notin [a - z]\}$$

3. **Length Filtering:** Discard any tokens where $\text{length}(w) < 3$.

**Step B: Morphological Grouping (Prefix/Suffix)** The algorithm identifies a "Base Word" (the category) and "Clue Words" (the members) by checking for valid compound formations.

1. **Prefix Logic:** For a target base $B$, find words $W$ in the vocabulary that end with $B$.
2. **Fragment Extraction:** Extract the prefix $P$ such that $W = P + B$.

3. **Constraint Validation:** $P$ is valid only if:
   - $P$ is itself a valid word in the vocabulary (length 2–8).
   - $P$ is not in the `BANNED_PREFIX_FRAGMENTS` list (e.g., "un", "re", "pre") to avoid trivial grammatical matches.
   - $z(P) \geq 3.0$ and $z(W) \geq 2.5$.
4. **Suffix Logic:** Mirrors the above, finding $W$ starting with $B$, where $W = B + S$, ensuring $S$ is not a common grammatical inflection like "-ness" or "-tion".

**Step C: Phonetic Grouping (Rhymes)**

1. Select a base word $B$.
2. Query the *CMU Pronouncing Dictionary* for all words sharing the same terminal phonemes (stress-dependent).
3. **Intersection:** Filter the resulting set against the cleaned $100,000$-word vocabulary.
4. **Quality Control:** Discard $B$ from its own rhyme set and ensure all candidates meet the Zipf threshold $z \geq 3.0$.

**Step D: Puzzle Generation and Selection**

1. **Sampling:** For any identified group of valid candidates $> 4$, the system uses a Random Number Generator (RNG) to sample exactly 4 words.
2. **Deduplication:** A unique key is generated for every group:

$$\text{Key} = \text{Type} \mid \text{Category} \mid \text{SortedMembers}$$

   This prevents the same puzzle from appearing in the output.
3. **Output Generation:** The system iterates until $1,600$ categories (800 morphological, 800 rhyming) are successfully generated.

**5. User-Provided Parameters**

- `n_clues`: Number of words per group (Default: 4).
- `min_len`: Minimum character count for words (Default: 3).
- `max_attempts`: Search ceiling to prevent infinite loops in sparse phonetic spaces.
- `rng`: A seedable random state (e.g., `Random(42)`) to ensure reproducibility of the exact same puzzle sets.

6. **Reproduction Instructions** To reproduce the results:

   1. Install requirements: `pip install wordfreq pronouncing`.
   2. Initialize the vocabulary using `top_n_list('en', 100000)`.
   3. Run the `build_before_after_categories` function to generate prefix/suffix groups.
   4. Run the `build_rhyme_categories` function using the `pronouncing.rhymes` method.
   5. Export the results to a JSON structure for verification.

## 3.5 Purple Words Other

1. **External Code and Data References**
   The system relies on the following external resources:

   - **Libraries:** `nltk` (Natural Language Toolkit) for lexical database access, `wordfreq` for Zipf-scale frequency analysis, and `pronouncing` for CMU dictionary phonetics.
   - **Datasets:** WordNet (via NLTK) provides the primary word pool. The script clones the `Eyefyre/NYT-Connections-Answers` repository to retrieve `connections.json`, which contains historical puzzle data for deduplication.

2. **Storage and Execution**
   The implementation is stored in a single script named `purple_gen.py`. To execute the generator with the default parameters (10,000 categories), run:

   ```
   python purple_gen.py
   ```

   Custom configurations, such as a smaller batch size or a specific frequency threshold, can be executed via:

   ```
   python purple_gen.py --count 500 --min-zipf 2.5
   ```

3. **Lexical Acquisition**
   The system utilizes the WordNet corpus to extract all base lemmas. Let $L$ be the set of all lemmas. We define the filtered pool $P$ as:

   $$P = \{w \in L \mid \text{is_alpha}(w) \wedge 3 \leq |w| \leq 12 \wedge \zeta(w) \geq \text{threshold}\}$$

   where $\zeta(w)$ is the Zipf frequency retrieved from the `wordfreq` library.

4. **Structural Heuristics**
   For every $w \in P$, the algorithm computes several boolean properties:
   - **Alternation:** $w$ is alternating if $\forall i \in [0, |w| - 2], \text{is_vowel}(w_i) \neq \text{is_vowel}(w_{i+1})$.
   - **Keyboard Constraints:** $w$ is "Left-Handed" if $\forall c \in w, c \in \{q, w, e, r, t, a, s, d, f, g, z, x, c, v, b\}$.
5. **Phonetic Analysis**
   Using the CMU Pronouncing Dictionary, words are mapped to phoneme strings. A *Homophone Group* is defined as:

   $$G_h = \{w \in P \mid \text{phonemes}(w) = p\}$$

   A category is generated by sampling 4 pairs of words $(a_i, b_i)$ such that $a_i$ and $b_i$ are homophones, then presenting the set $\{a_1, a_2, a_3, a_4\}$ to the user.
6. **Hidden Word Matrix**
   The generator constructs a mapping $M : h \to \{w_1, w_2, \ldots, w_n\}$ where $h$ is a substring of $w$. To generate a "Hidden Rhyme" category, the algorithm finds a set of hidden words $H = \{h_1, h_2, h_3, h_4\}$ such that:

   $$\forall h \in H, \text{RhymeKey}(h) = K$$

   The final members are chosen by sampling one $w$ from each $M(h_i)$.
7. **Deduplication Logic**
   Given a candidate group $C = \{w_1, w_2, w_3, w_4\}$, and the set of historical NYT categories $S_{past}$:

   $$\text{Valid}(C) \iff \text{sort}(C) \notin S_{past} \wedge |\{w_1, w_2, w_3, w_4\}| = 4$$

8. **Output Generation**
   The resulting valid categories are serialized to a JSON format containing the `group` name (the connection logic), `members` (the 4 words), and a detailed `explanation` for the user.

## 3.6 Puzzle assembly and Red Herring detection

- **Technical Appendix: Automated Connections Puzzle Generation**

  This appendix details the computational methodology, mathematical foundations, and replication steps for the Connections generation system.

- **Implementation Details and File Location**
  - **Primary Script:** clustering_code.ipynb

- **Environment:** The code is designed to run in a Jupyter Notebook environment (e.g., Google Colab or local Jupyter server).
- **Model Architecture:** Semantic processing is handled by the `all-MiniLM-L6-v2` Sentence Transformer, a 384-dimensional dense vector model optimized for clustering and semantic search.

- **External Data Sources and Dependencies**
  - **Data Files:** The system requires four JSON schema-specific files: `Yellow Data.json`, `green_categories.json`, `blue_trivia.json`, and `purple_categories.json`.
  - **Code Libraries:**
    * `sentence-transformers`: BERT-based embedding generation.
    * `scikit-learn`: Implementation of `AgglomerativeClustering` and `cosine_similarity`.
    * `numpy` & `pandas`: Matrix manipulation and data framing.

- **Conceptual and Mathematical Foundations**
  - **Hybrid Vector Embedding:** Rather than embedding only the category label, we compute a Hybrid Vector ($V_{hybrid}$) to represent the category's total semantic footprint. This is calculated as the normalized weighted sum of three distinct vectors:

    $$V_{hybrid} = \text{norm}(0.55 \cdot V_{combo} + 0.30 \cdot V_{centroid} + 0.15 \cdot V_{label})$$

    Where $V_{combo}$ is the embedding of the group and members combined, $V_{centroid}$ is the mean of the four member word vectors, and $V_{label}$ is the label alone.
  - **Cosine Similarity Metric:** To determine the "Red Herring" potential between categories, we calculate the similarity between any two hybrid vectors $\mathbf{A}$ and $\mathbf{B}$ using the dot product of normalized vectors:

    $$\text{similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

    The algorithm prioritizes candidates where this value is maximized, ensuring thematic overlap.

- **Step-by-Step Computational Workflow**
  1. **Initialization:** Install dependencies and load the Sentence Transformer model into memory.
  2. **Schema Normalization:** Data is loaded from JSON and mapped to a standard `Category` dataclass. Text is converted to uppercase and validated to ensure exactly four members per group.

3. **Vectorization:** For every category in the bank, the script generates the $V_{hybrid}$ representation.
4. **Puzzle Construction Algorithm:**
   - **Anchor Selection:** A category is randomly sampled from the Purple (wordplay) bank.
   - **Similarity Search:** The script executes a cross-color neighbor search to find the Blue category with the highest cosine similarity to the Purple anchor.
   - **Iterative Chaining:** The process repeats: Green is selected based on proximity to Blue, and Yellow is selected based on proximity to Green.
5. **Clustering Analysis:** Using `AgglomerativeClustering` with a `cosine` metric and `average` linkage, the script groups similar categories across the entire bank to identify recurring themes.

- **Replication and Execution Instructions** To reproduce the results without the provided notebook file:
  1. Populate a directory with the four required JSON data files.
  2. Initialize a Python environment and install the listed libraries.
  3. Implement the `preprocess_categories` logic to generate weighted hybrid embeddings for all categories.
  4. Run a nearest-neighbor search using `np.dot()` on the normalized hybrid vectors to find overlapping groups between different color difficulty tiers.

## 3.7 Front End

**1. Project Overview and File Structure** This system automates the creation of "Connections" style puzzles by selecting four distinct categories (sets of 4 words) from a precomputed database. The pipeline is split into two phases:

- **precompute.py**: A data processing script that transforms raw JSON word sets into a searchable vector database with cross-color similarity mappings.
- **generator.py**: A logic engine that utilizes precomputed similarities to assemble a cohesive 16-word puzzle board, ensuring semantic challenge and word uniqueness.

**2. External Data and Libraries** To reproduce these results, the following dependencies and data sources are required:

- **External Libraries**:
  - `sentence-transformers`: Used for generating semantic embeddings (specifically the `all-MiniLM-L6-v2` model).

- `numpy`: For vector mathematics (centroids and dot products).
- `scikit-learn` (optional): For advanced similarity metrics, though dot product is used here.

- **Data Sources**: The scripts expect four JSON files (`yellow.json`, `green.json`, `blue.json`, `purple.json`). Each file must contain objects with a group name (category) and a list of 4 member words.

**3. Mathematical Definitions and Concepts**

- **Sentence Embeddings**: We represent text as a dense vector $v \in R^n$. Using `all-MiniLM-L6-v2`, each string is mapped to a 384-dimensional space where spatial proximity correlates to semantic similarity.

- **Vector Normalization**: Every vector is normalized to unit length ($L_2$ norm) to ensure that the dot product is equivalent to Cosine Similarity:

$$\hat{v} = \frac{v}{\|v\|_2}$$

- **Centroid Calculation**: The representative vector for a set of words is the mean of their individual vectors, re-normalized:

$$C = \text{normalize}\left(\frac{1}{n}\sum_{i=1}^{n}\hat{v}_i\right)$$

- **Cosine Similarity**: The similarity $S$ between two categories $i$ and $j$ is computed as the dot product of their hybrid vectors:

$$S(i,j) = \hat{v}_i \cdot \hat{v}_j$$

**4. Algorithmic Step-by-Step: Precomputation (`precompute.py`)**

- **Step 1: Schema-Aware Loading**: The script parses raw JSON. It enforces a strict "4-word rule"—any category that does not have exactly 4 unique members is discarded.

- **Step 2: Hybrid Embedding Generation**: For each category, three distinct embeddings are created:

  1. $V_{label}$: Embedding of the category name (e.g., "TYPES OF SHOES").
  2. $V_{combo}$: Embedding of the string "Category: Word1, Word2, Word3, Word4".
  3. $V_{members}$: The centroid of the 4 individual word embeddings.

  A final **Hybrid Vector** is computed using a weighted average to prioritize context:

$$V_{hybrid} = 0.55(V_{combo}) + 0.30(V_{members}) + 0.15(V_{label})$$

- **Step 3: Cross-Color Neighbor Search**: For every category $i$, the script iterates through all categories $j$ of a *different* color. It calculates $S(i, j)$ and stores the top $k$ (default 10) most similar categories. This creates "red herrings" or thematic overlaps between difficulty tiers.

**5. Algorithmic Step-by-Step: Generation (`generator.py`)**

- **Step 1: Purple Selection**: The engine starts by picking a random "Purple" (hardest) category. This anchors the puzzle's difficulty.
- **Step 2: Similarity-Based Neighboring**: Instead of picking three more random groups, the engine retrieves the precomputed "neighbor" for the Purple group. This ensures that at least two groups in the puzzle share a semantic theme, increasing the difficulty of the "Connections" logic.
- **Step 3: Disjoint Set Validation**: As groups are added, the engine maintains a `used_words` set. Any candidate group whose words overlap with already selected words is rejected:

$$\text{WordSet}_{new} \cap \text{WordSet}_{used} = \emptyset$$

- **Step 4: Filling the Board**: The engine picks random categories for the remaining two colors that satisfy the disjoint set requirement.
- **Step 5: Randomization**: The final 16 words are flattened and shuffled using a Fisher-Yates shuffle algorithm to remove any trace of their original groupings before being sent to the front-end.

**6. User Parameters**

- `--sample-per-color`: Limits the database size to speed up precomputation.
- `--top-k`: Determines how many potential "neighbors" are stored. A higher $k$ provides more variety but may lower the average similarity of the pairs.
- `max_tries`: In the generator, this defines how many times the script will attempt to find 4 non-overlapping groups before failing.

**7. Execution Guide** To reproduce the processed data and run a local generation:

1. **Install Dependencies**: `pip install sentence-transformers numpy`
2. **Run Precomputation**:

```
python precompute.py --yellow yellow.json --green green.json \
                     --blue blue.json --purple purple.json \
                     --output processed/processed_categories.json
```

3. **Generate Puzzle**: Run `python generator.py`. The script will load the JSON from the `processed/` directory and output a 16-word board and its solution key to the console.